MATERIALIZE — Future Directions & Fixes

A ranked roadmap of the extensions, fixes, and research directions discussed for MATERIALIZE. Companion to `MATERIALIZE.md`. Each item lists **why it matters**, **what to do**, its **size** (rough developer-time), and its **dependencies / risk**.

Size key (single competent developer familiar with the codebase): `S` ≈ 1–3 days · `M` ≈ 1–2 weeks · `L` ≈ 3–6 weeks · `XL` ≈ 2+ months (research-grade).

Ranking = importance (scientific/user value) balanced against tractability. Top of the list = do first.

#	direction	value	size	touches protected solver?
1	Incompatible reference metric → **residual stress** (add $\delta\bar{g}$ source)	★★★★★	**M–L**	yes
2	**Stability / anti-mechanism** constraint in the design loss	★★★★☆	**S–M**	no
3	Differentiable **open-boundary** (cut-and-stretch) forward	★★★★☆	**M**	partial
4	**Connectivity / topology** design (sparsify k)	★★★★☆	**M**	no
5	**Cluster** (exact) forward model for the hard regimes	★★★☆☆	**L**	new module
6	Solution-manifold analysis (multi-start **clustering**)	★★★☆☆	**S–M**	no
7	Curvature/angle-KKT **numerical hardening**	★★★☆☆	**S–M**	yes
8	**Nonlinear / large-deformation** constitutive	★★★☆☆	**XL**	yes
9	Phase 4 **ML surrogate + generative** inverse design	★★★☆☆	**XL**	no
10	η=0.5 **second-order** ($O(\delta^2)$) homogenisation correction	★★☆☆☆	**M**	yes
11	**3D** extension	★★★★☆	**XL**	rewrite
12	Docs / demo / packaging polish	★★☆☆☆	**S**	no
13	**Differentiable positions** → simultaneous k+position design	★★★☆☆	**M–L**	yes

---

1 — Incompatible reference metric → residual stress · ★★★★★ · L

The single highest-value extension. Today the framework designs a *stress-free* target ($\bar{g} = I$). Real frustrated metamaterials (growth-patterned gels, prestressed shells, buckling sheets) carry **residual stress** from a **non-Euclidean reference metric** — and the D2C metric formulation is already written in exactly that (incompatible-elasticity) language, so this is a natural, high-payoff generalisation. See `MATERIALIZE.md` §4 for the theory, ready.

The mechanism (corrected). Let the reference fluctuate on the patch scale, $\bar{g}(s) = \bar{g} + \delta\bar{g}(s)$, keeping the response ansatz $\delta g(s) = W(s)\,\Delta g$. The operators are **unchanged**: the edge carrier q_e, the bare tensor $A(s)$, and the edge/curvature/mean constraints (which act on the *actual* field δg, still homogeneous) all stay as they are — the reference's incompatibility does **not** go on a constraint RHS. The single change is the substitution $(\Delta g + \delta g) \to (\Delta g + \delta g - \delta\bar{g})$ in the energy, so $\delta\bar{g}$ enters as an added **source** $A(s)\,\delta\bar{g}(s)$ on the RHS of the same block solve; $\delta\bar{g} = 0$ recovers the current equation exactly. See `MATERIALIZE.md` §4.2 and the indexed working note `residual_stress_note.pdf`.

The blocker. The current solver has **no $\delta\bar{g}$ input** — the reference is pinned to the actual geometry ($\bar{g} = I$), and rest length enters only through k_e/ℓ_e^2, so ℓ_0-design *appears* degenerate with k-design (a flat-gauge artefact — $\bar{g} = \bar{g}(\ell_0)$ in general). The fix is to (i) supply $\delta\bar{g}(s)$, (ii) add the source $A(s)\,\delta\bar{g}(s)$ to the RHS, (iii) report the prestress from the mismatch $\sigma(s) = A(s)(\Delta g + \delta g - \delta\bar{g})$. The genuinely open problem is **not** the mechanics but the **self-consistent definition of $\delta\bar{g}$** (the patch-scale split of a prescribed reference field) and the finite-base-strain treatment — both under active discussion, deferred.

Open sub-problem — the reference-metric split (a correct finite-size theory, not a convention pick). Defining $\delta\bar{g}$ is upstream of implementing it. The decomposition $\bar{g} = \bar{g}_{\mathrm{bg}} + \delta\bar{g}$ has **two independent axes**: a **scale filter** (macroscopic Δg vs sub-patch reference) and a **compatible/incompatible (St-Venant, inc) projection** — only the incompatible part $\mathrm{inc}(\bar{g}) \neq 0$ sources stress. In the current **flat** setting the second axis collapses to $\bar{g}$ **constant vs non-constant** (any *compatible* $\bar{g}$ is flattenable to constant by a coordinate choice); the full compatible/incompatible distinction earns its keep only for non-flat / higher-D embeddings (#11). Candidate backgrounds: **(A)** uniform patch-mean [perturbative default], **(B)** systematic-vs-disorder, **(C)** spectral / scale-cutoff, **(D)** compatible/incompatible projection, **(E)** energy-optimal compatible projection, **(F)** ensemble mean (unifies A/B). The correct split + its finite-size corrections — especially **ordered finite-size curvature**, where scales do *not* separate and not everything belongs in $\delta\bar{g}$ — is a genuine theory task, upstream of the $\delta\bar{g}$ source below.

Why it still touches the protected core: it modifies `forward_solver_torch.py` (a new RHS source + a prestress output), needs its own physical ground truth (a prestressed-network virial with residual stress) and regression tests, and the inverse engine needs a `'prestress'` objective kind. But it is an *added source term*, **not** a rebuild of the operators — lighter than first framed; the metric machinery already does the rest.

Deliverables: a $\delta\bar{g}$ input + `prestress` output/objective (ℓ_0 non-degenerate); a residual-stress demo (e.g. a target disclination pattern / a self-buckling sheet) verified against a prestressed simulation.

---

2 — Stability / anti-mechanism constraint · ★★★★☆ · S–M

Why: the recurring failure mode across the demos (high-contrast rings, strong bulges, strongly auxetic patches) is the optimiser drifting to a **near-mechanism** — a design with a near-zero stiffness eigenvalue where the linear readback diverges from the true nonlinear response. We currently *detect* this after the fact (independent nonlinear check) and *discourage* it indirectly (`homogeneity` penalty, `reg`). A direct **stability term** would prevent it.

What to do: add a differentiable penalty on the smallest eigenvalue of the design's stiffness (or of the per-region response Hessian) — e.g. a soft-plus barrier `-λ·min_eig` or a penalty on the condition number of the KKT solve — as an opt-in `Objective('stability', ...)` or a global option in `optimize`. Cheap eigen-estimates (a few Lanczos/power iterations on the sparse operator) keep it differentiable and fast.

Size: S if a coarse penalty on `min(k)` -like proxies suffices; M for a proper smallest-eigenvalue barrier with its own test. No protected-core change (it lives in the loss).

---

3 — Differentiable open-boundary (cut-and-stretch) forward · ★★★★☆ · M

Why: the periodic homogenised response and the **real open-boundary** response can differ (edge effects, Eshelby domination for small inclusions) — we saw this repeatedly (the ribbon, the inclusion, the bulge). Today the open cut-and-stretch (`_common.open_stretch` , a classical nodal spring solve) is used only for *verification*; it is plain NumPy and not autograd-connected. Making it differentiable would let us **design directly for the specimen** a user will actually build.

What to do: re-implement the open-boundary spring relaxation (`spring_K` , boundary conditions, sparse solve) in torch with an adjoint (mirror the Phase 2 large-N adjoint pattern), and expose it as an alternative `DesignProblem.forward_open(...)` that the inverse engine can target.

Size: M — it is a self-contained sparse linear solve + adjoint; the physics is simpler than the intrinsic solve. Partially touches the design plumbing (a second forward path) but not the protected periodic solver.

---

4 — Connectivity / topology design (sparsify k) · ★★★★☆ · M

Why: the current engine tunes stiffnesses on a **fixed** connectivity. Allowing bonds to be *removed* (soft-thresholded to $k = 0$ on a dense base graph) unlocks genuine topology design — lattices, re-entrant honeycombs, chiral cells — and often reaches responses (strong auxetics, mechanisms-on-purpose) that a fixed graph cannot.

What to do: add an L1 / L0-surrogate sparsity penalty on k (or a continuous "bond present" gate), with a schedule that anneals bonds off; post-process to a discrete graph and re-verify. Needs care so removed bonds do not create *unintended* mechanisms (couple with #2).

Size: M — lives entirely in the inverse engine (a new penalty + an anneal-and-prune loop + a "realise the sparse graph" step). No protected-core change.

---

5 — Cluster (exact) forward model for the hard regimes · ★★★☆☆ · L

Why: the intrinsic metric solve reproduces the simulation to 3–4 digits across most of parameter space, but degrades in two regimes: very strong disorder ($\eta \gtrsim 0.5$) and very strong rigidity contrast (long correlation length). `THEORY_NOTES.md` shows the cure is a **cluster** forward model — relax a small node patch around each triangle (boundary held affine, actual local physics) and read the central triangle's response. It is exact (works in node space ⇒ automatically compatible; uses the real local neighbours), non-iterative, loading-independent, and embarrassingly parallel.

What to do: implement the per-triangle cluster solve (one small dense linear solve each, batched/parallel), with an adjoint for gradients; select cluster radius ≈ the local correlation length. Offer it as an alternative `method='cluster'` for the hard regimes / final refinement.

Size: L — a genuinely new solver module (geometry of the patches, the batched solve, the adjoint, tests vs the sim at high η/contrast). High scientific value where the mean-field/intrinsic solve is weakest, but most day-to-day design already sits in the regime where the intrinsic solve is exact.

---

6 — Solution-manifold analysis (multi-start clustering) · ★★★☆☆ · S–M

Why: the inverse problem is heavily underdetermined ($\sim 3N_{\text{bond}} \to 6$) — for one target there is a *manifold* of valid k. `optimize` already has `n_restarts` but only keeps the best. Clustering the restarts (and per-edge coefficient-of-variation) would map the solution manifold, reveal which bonds are essential vs free, and surface *distinct* design families for the same target.

What to do: run many restarts, cluster the resulting $\boldsymbol{k}$-vectors (e.g. by response fingerprint or per-edge stats), report representative designs + a "which bonds matter" map.

Size: S–M — pure post-processing on top of the existing `optimize`; no solver change.

---

7 — Curvature/angle-KKT numerical hardening · ★★★☆☆ · S–M

Why: the curvature (vertex-angle) constraint block can go **near-singular** on meshes with near-degenerate (sliver) triangles — the constraint Gram loses rank and the saddle solve needs a larger multiplier regularisation, costing accuracy. `THEORY_NOTES.md` flags this as the fragile part of the intrinsic solve.

What to do: robustify the angle-gradient assembly near degeneracies (clamp/limit, or drop rank-deficient rows via a stable null-space detection), and/or pre-condition the KKT saddle. Add tests on deliberately slivered meshes.

Size: S–M. Touches the protected solver's constraint assembly, so it needs the full regression + physical suite.

---

8 — Nonlinear / large-deformation constitutive · ★★★☆☆ · XL

Why: the framework is geometrically exact but **linear in energy** (Section 3.2) — no strain-stiffening, snap-through, or finite-strain constitutive nonlinearity. Many real metamaterial effects (programmable buckling, multistability) live there.

What to do: replace the quadratic per-triangle energy with a finite-strain spring energy and solve the (now nonlinear) metric equilibrium (Newton on the KKT), with sensitivities via the implicit function theorem. This is a substantial change to the core physics and to differentiability.

Size: XL — research-grade; touches the protected core deeply and needs a new verification story. Consider *after* #1 (residual stress), since a curved reference + nonlinear energy is where the most interesting frustrated-metamaterial physics lives.

---

9 — Phase 4: ML surrogate + generative inverse design · ★★★☆☆ · XL

Why: for very large libraries or real-time design, a learned **GNN forward surrogate** (~1 ms vs the solver's ms, with ~2% error) and a **conditional VAE** generative inverse (sample *many* distinct designs for one target) complement the exact optimiser. The theory/architecture is written in `Tutorial.md` (NNConv message passing, Set2Set readout, CVAE with a physics loss, two-phase surrogate→solver training).

What to do: build the dataset (designs + verified responses), train the forward GNN, then the CVAE with a physics-consistency term (evaluate generated designs through the solver). Largely a separate track from the core solver.

Size: XL — a full ML pipeline (data, training, evaluation), but self-contained (no core change).

---

10 — η=0.5 second-order homogenisation correction · ★★☆☆☆ · M

Why: the intrinsic solve matches the simulation to first order in the strain amplitude; at extreme disorder (η=0.5) the $O(\delta^2)$ term of the exact area-conservation law becomes visible (`INTRINSIC_METRIC_SOLVE.md` §7–8, residual $\sim 3 \times 10^{-3}$). Carrying the next order (or using the cluster, #5) removes it.

What to do: add the second-order area-law term to the closure, or fall back to the cluster in that regime. Mostly of theoretical interest — everyday design does not reach it.

Size: M; touches the protected solve. Low priority.

---

11 — 3D extension · ★★★★☆ · XL

Why: the entire framework is 2D (triangles, plane-elasticity Voigt-3). Real fabricated metamaterials are often 3D (tetrahedral spring networks, plate/shell assemblies). A 3D version would be a major scientific and practical step.

What to do: generalise the metric (6-component in 3D), the bare-tensor assembly (tetrahedra), the compatibility constraints (edge + the 3D curvature/incompatibility tensor), and the homogenisation (6×6 C_{eff}). Essentially a re-derivation and re-implementation of Phases 2–3.

Size: XL — a new project on the same principles. High value, high cost; sequence after the 2D theory is complete (#1) so the 3D version inherits residual stress from the start.

---

12 — Docs / demo / packaging polish · ★★☆☆☆ · S

Ongoing: a couple of schematic vector diagrams (the KKT block structure; the metric-vs-displacement picture) rendered natively; a `pip` -installable package layout; a one-command "reproduce all figures" script; expanding the worked-example gallery. Low risk, incremental.

---

13 — Differentiable positions → simultaneous k+position design · ★★★☆☆ · M–L

Why: node positions are a large design lever, but today they are optimised **separately** from k: `designer.design` searches k over topologies, then polishes the top design by **alternating** [gradient k-design] ↔ [derivative-free **SPSA** position nudges]. SPSA is used because positions have **no gradient through the solver** — they enter via the geometry (edge carriers q_e, triangle areas, the compatibility/curvature/mean constraint operators), which is off the autograd path (only k, ℓ_0 are differentiable). Alternating is the pragmatic compromise (fast gradient-k + slow SPSA-positions).

What to do: make the solver **differentiable w.r.t. node positions** — differentiate the geometry assembly (q_e , areas) and the constraint operators w.r.t. node coordinates — so k and positions can be optimised **simultaneously** by gradient descent on one joint objective, replacing the alternating SPSA polish. Touches the protected core (new gradients through the geometry).

Payoff / caveat: positions become a first-class gradient DOF; but the empirical anisotropy-amplitude ceiling is a topology/size bound (positions won't beat it — see the design-workflow conventions), so the gain is on *reachable* targets, not the extreme-anisotropy frontier.

Already done (for reference)

- **Differentiable large-N adjoint** — gradients through the intrinsic solve above 600 triangles (~1.07× cost); design runs to ~16k triangles.
- **Physical (unweighted) homogenisation fix** — the effective tensor is the unweighted physical mean (energy = virial), replacing the legacy area-weighted metric average.
- **Strain / stress objectives + homogeneity regulariser** — target the actual per-triangle strain/stress under any load, plus a within-region uniformity penalty (Section 7.4–7.5).
- **The verification harness** — every design independently simulated (virial/energy), with the `strain_stress`, `two_region`, `auxetic_patch`, `anisotropy`, and mode-selective demo suites.